

2023 fall 《计算机系统结构》中文班 考试指南

一、考试成绩占比

- 平时+作业或其他 50%
- 期末考试 50 %

二、知识点总结

1、概述与性能

- 性能评价指标：带宽、延迟
- CPU 执行时间公式；Amdahl 定律

2、整数

- 补码的编码和特点
- 无符号整数、带符号整数之间的转换
- 定点数的逻辑移位、算术移位、扩展、截断
- 补码加减运算溢出的判断

3、浮点数

- 浮点数的表示方法
- 浮点数的范围和精度
- 浮点数运算结果的舍入方法

4、指令系统

- CISC 和 RISC
- RISC-V 指令格式与编码
- 函数调用、栈在机器语言中的实现

5、处理器设计

- CPU 内部结构、指令的执行过程、数据通路基本组成
- 单周期处理器的控制器设计

6、指令流水线

- 指令流水线基本原理、
- 流水段寄存器概念、流水线数据通路设计
- 流水线控制信号
- 各类冒险及其处理技术
- 控制冒险、动态转移预测

7、存储系统

- cache 的基本原理和映射方式（全相联、组相联、直接映射）
- cache 更新策略和替换策略
- 影响 cache 性能和命中率的关键因素

8、其他（掌握概念，只考选择题或者概念简答题，不考分析题）

- 多发射、静态调度、动态超标量、按序多发射流水线（按序超标量）、乱序超标量处理器
- 数据级并行开发（SIMD）的工作原理及相关概念
- 多核处理器的高速缓存一致性、假共享的概念

复习题（高速缓存）

1、直接映射（directed mapped）

为了将内存数据块放置到高速缓存中，可以将内存地址分几部分看待：

Tag bits	Index bits	Offset bits
----------	------------	-------------

Offset bits 是块内偏移，Index bits 用于标记该内存数据块在高速缓存中的组号(set number)，剩下的 tag bits 是该数据块的标记。

如果高速缓存采用直接映射（direct mapped）采用写直达（write through）的更新策略，那么高速缓存中每一行包含的内容为：cache data block, tag, valid bit，不需要包含 dirty bit.

根据已知信息，请填写下表中的空格部分：

Address size(bits) 内存地址的长度, 寻址到 byte	Cache size	Block Size 数据块大小	tag bits tag 位数	Index bits Index 位数	Offset bits 块内偏移位数	Bits per row 每一行的总位数
16	4KB	4B	4	10	2	32+4+1
32	32KB	16B				
32			16	12		
64	2048KB			14		1068

答案：

Address size (bits)	Cache size	Block size	Tag bits	Index bits	Offset bits	Bits per row
16	4KiB	4B	4	10	2	32+4+1
32	32KiB	16B	17	11	4	128+17+1
32	64KiB	16B	16	12	4	128+16+1
64	2048KiB	128B	43	14	7	1068

2、组相联映射(set associative)：

假设某计算机的主存地址空间大小为 64MB，采用字节编址方式。其 cache 数据区容量为 4KB，采用 4 路组相联映射方式、LRU 替换和回写（write back）策略，块大小为 64B。请问：

(1) 主存地址字段如何划分？要求说明每个字段的含义、位数和在主存地址中的位置。

(2) 该 cache 的总容量（不仅包括数据区容量）有多少位？

参考答案：

(1) cache 的划分为： $4KB = 2^{12}B = 2^4 \text{ 组} \times 2^2 \text{ 行/组} \times 2^6 \text{ 字节/行}$ ，所以，cache 组号（组索引）占 4 位。主存地址划分为三个字段：高 16 位为标志字段（1 分）、中间 4 位为组号（1 分）、最低 6 位为块内地址

即主存空间划分为： $64MB = 2^{26}B = 2^{16} \text{ 组群} \times 2^4 \text{ 块/组群} \times 2^6 \text{ 字节/块}$

(2) cache 共有 64 行，每行中有 16 位标志、1 位有效位、1 位修改(dirty)位、2 位 LRU 位，

以及数据 64B。故总容量为 $64 \times (16 + 1 + 1 + 2 + 64 \times 8) = 34048$ 位。

3. 假设有一个 2 路组关联映射的高速缓存

- 字(word) 的大小为 4bytes
- 访存地址寻址到字节
- 每一个高速缓存块(block) 大小为 512 bytes
- cache 总共有 1024 块 (blocks)

(1) 假设访问内存的物理地址为 32 位、cache 在初始时空，一次内存访问会将多少个字(word)读入 cache? (2 分)

Answer

- The entire block will be filled
- If words are 4 bytes long and each block holds 512 bytes, there are $2^9 / 2^2$ words in the block
- i.e. there are 2^7 or 128 words in each block

(2) 如果访问物理地址 F A B 1 2 3 8 9 (十六进制)，数据会被放入 cache 中第几个 set (组) (4 分)

Answer

We need to determine what the index bits are. From above, we know the offset is 9 bits (remember, data is byte addressable) – so we will need to break up the hex address into binary:

1111 1010 1011 0001 0010 0011 1000 1001

Our offset for this address is: 1 1000 1001

$1024 / 2 = 2^{10} / 2^1 = 512 = 2^9$ – therefore 9 bits of index are required.

These are: 01 ■ 0010 ■ 001 which implies the address maps to the 145th set.

4. Consider a unified, 64 Kbyte, direct mapped, instruction and data cache that exhibits the following miss rates as a function of block size, keeping the cache size fixed.

Block Size	Miss Rate
4	35.18%
8	22.57%
16	16.009%
32	12.47%
64	10.79%
128	10.27%
256	10.72%
512	12.75%
1K	16.41%
2K	23.15%
4K	32.97%

a. Why does the miss rate initially decrease as the block size increases? Why does it eventually start increasing again?

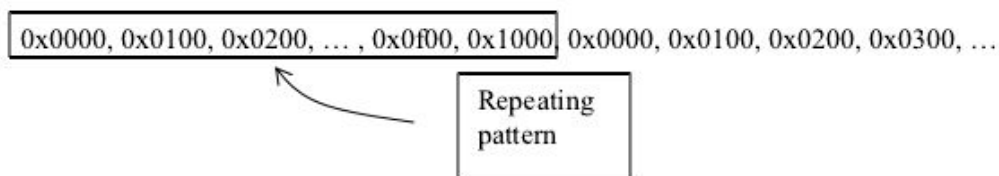
Initially increased spatial locality causes a decrease in miss rate that is subsequently overrun by decreased temporal locality.

b. Can the 128 byte block size result be guaranteed to provide the best average memory access time? Justify your answer.

No. Smaller block sizes may have slightly higher miss rate (e.g., 64 byte blocks) that may be overshadowed by the much larger miss penalty of a 128 byte block. The answer depends on the design of the cache-memory interface. For average memory access time the key is the behavior of the miss penalty, $(1-h)*t_p$, where t_p is the time to fetch a block from memory and $(1-h)$ is the miss rate.

5. We have a 4 Kbyte cache operating with 256 byte lines. Consider two designs for this cache. The first is a direct mapped design. The second is a fully associative cache with an LRU replacement policy. Physical addresses are 16 bits. Provide a sequence of physical addresses (in hexadecimal notation) that when repeated indefinitely will result in a lower miss rate in the direct mapped cache than in the fully associative cache. The address stream should be accompanied by an explanation of this behavior.

The following stream of addresses will cause this to happen (the 0x designation signifies hexadecimal notation)



There are sixteen lines in the cache. Have a sequence of 17 addresses that address 17 consecutive blocks of memory and repeat itself. In a fully associative cache with an LRU policy after the cache as filled up with the first 16 addresses you get the following behavior. The least recently used line is always removed before it is referenced on the next memory reference. Thus, every reference becomes a miss.

In a direct mapped cache, the 17th and 1st reference will always miss but the remaining will always hit. In this case the rigidity of the direct mapped cache causes misses to be localized to two blocks while the flexibility of fully associative placement causes a miss every reference.

6. In the Table 1, we have given you four different sequences of addresses generated by a program running on a processor with a data cache. Cache hit ratio for each sequence is also shown below. Assuming that the cache is initially empty at the beginning of each sequence, find out the following parameters of the processor's data cache:

- Associativity (1, 2 or 4 ways)
- Block size (1, 2, 4, 8, 16, or 32 bytes)
- Total cache size (256 B, or 512 B)
- Replacement policy (LRU or FIFO)

Assumptions: all memory accesses are one byte accesses. All addresses are byte addresses.

Sequence No.	Address Sequence	Hit Ratio
1	0, 2, 4, 8, 16, 32	0.33
2	0, 512, 1025, 1536, 2048, 1536, 1025, 512, 0	0.33
3	0, 64, 128, 256, 512, 256, 128, 64, 0	0.33
4	0, 512, 1024, 0, 1536, 0, 2048, 512	0.25

Table 1: Sequences of Addresses

Cache block size - 8 bytes: For sequence 1, only 2 out of the 6 accesses (especially those to addresses 2 and 4) can hit in the cache, as the hit ratio is 0.33. With any other cache block size but 8 bytes, the hit ratio is either smaller or larger than 0.33. Therefore, the cache block size is 8 bytes.

Associativity - 4: For sequence 2, blocks 0, 512, 1024 and 1536 are the only ones that are reused and could potentially result in cache hits when they are accessed the second time. Three of these four blocks should hit in the cache when accessed for the second time to give a hit rate of 0.33 (3/9).

Given that the block size is 8 and for either cache size (256 B or 512 B), all of these blocks map to set 0. Hence, an associativity of 1 or 2 would cause at most one or two of these four blocks to be present in the cache when they are accessed for the second time, resulting in a maximum possible hit rate of less than 3/9. However, the hit rate for this sequence is 3/9. Therefore, an associativity of 4 is the only one that could potentially give a hit rate of 0.33 (3/9).

Total cache size - 256 B: For sequence 3, a total cache size of 512 B will give a hit rate of 4/9 with a 4-way associative cache and 8 byte blocks regardless of the replacement policy, which is higher than 0.33. Therefore, the total cache size is 256 bytes.

Replacement policy - LRU: For the aforementioned cache parameters, all cache lines in sequence 4 map to set 0. If a FIFO replacement policy were used, the hit ratio would be 3/8, whereas if an LRU replacement policy were used, the hit ratio would be 1/4. Therefore, the replacement policy is LRU.